



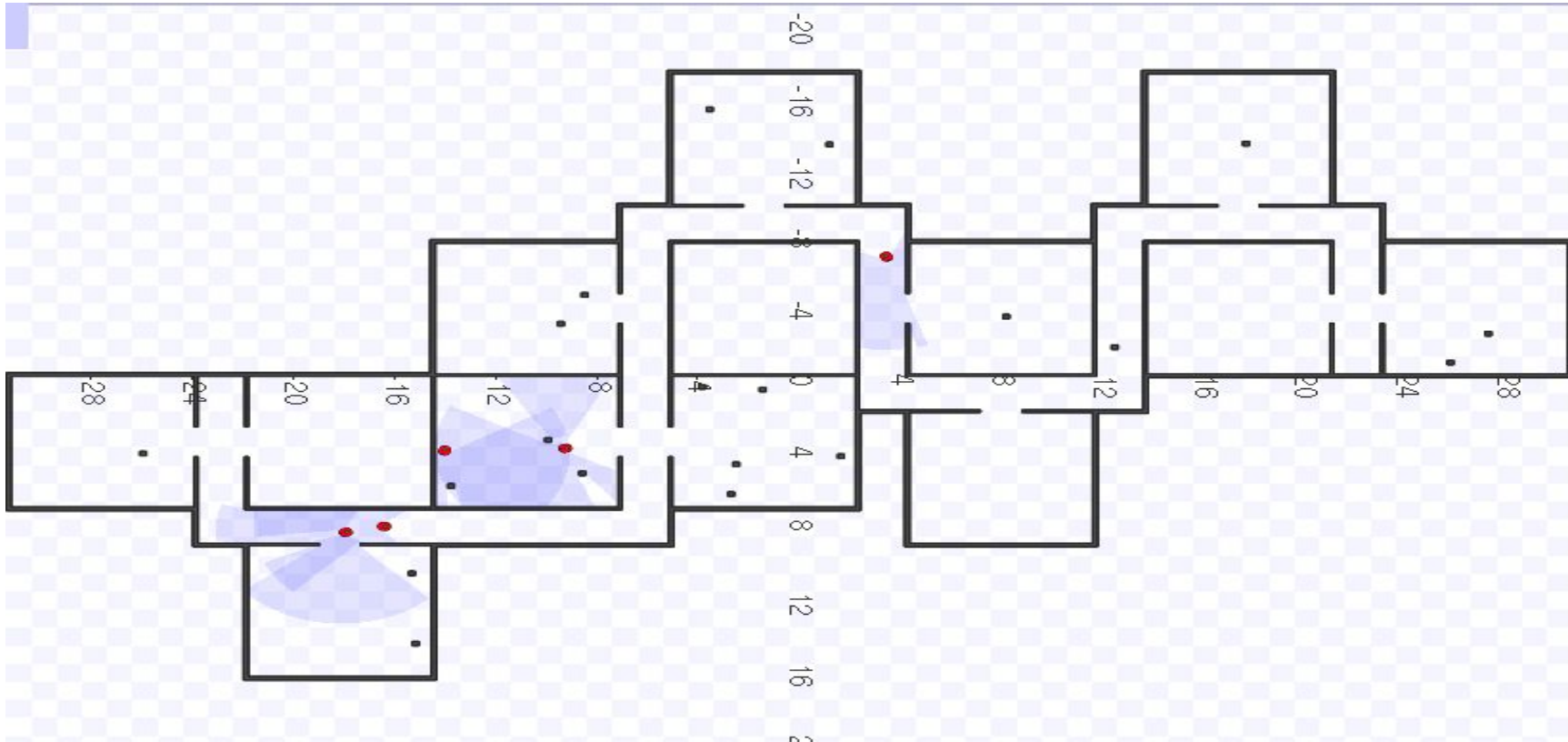
Oregon State
University

Comms-Constrained Multi-Agent Search

Liam Bouffard, Conor Gagliardi, Noah Pragin



Comms-Constrained Multiagent Search



Background

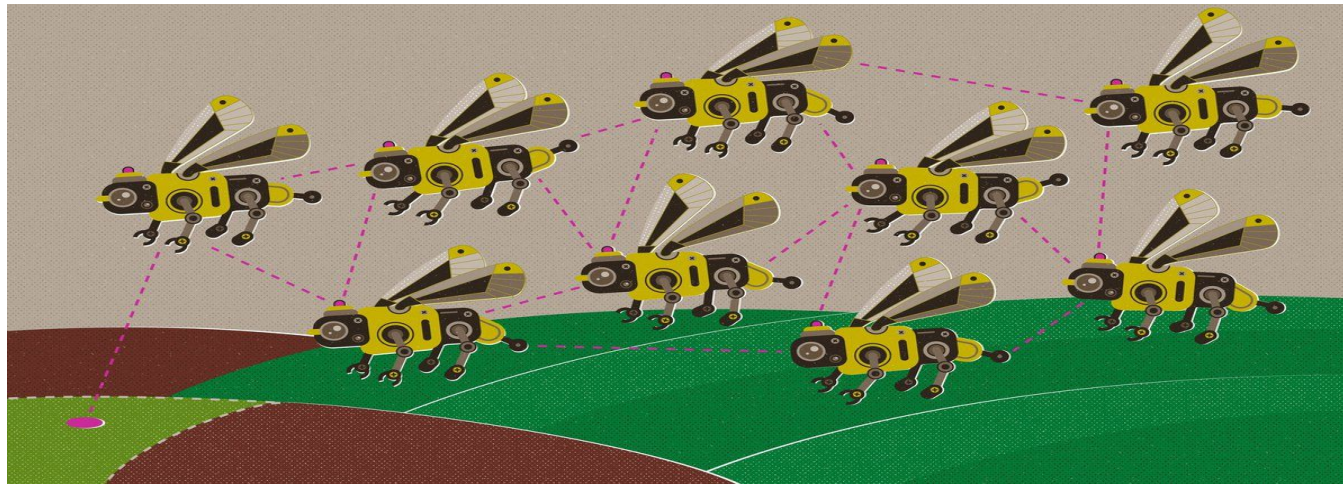
- Finding things is hard
- Extra help makes it less hard
- Multiagent applications in robotics
 - Add redundancy
 - Reduce time to mission success
 - Is as cool as a single robot times the number of robots in the system





Why did we choose this?

- Our team has preliminary experience when it comes to single agent robotics applications
- Preparing for Julie's class
- Applicable to our research / interest areas





Comms-Constrained Multiaгент Search

- In the real world, things are noisy
- Line of sight?
- The power of communication
 - Bluetooth, wifi, 5G?
- The powers that be which deny our robots freedom of communication
 - Cave, Ocean, Natural Disaster

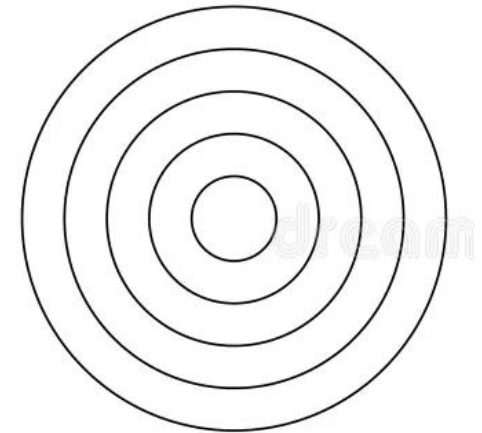




Comms-Constrained Multiaгент Search

Our approach

- Simulating limited bandwidth
- Planning required for information sharing

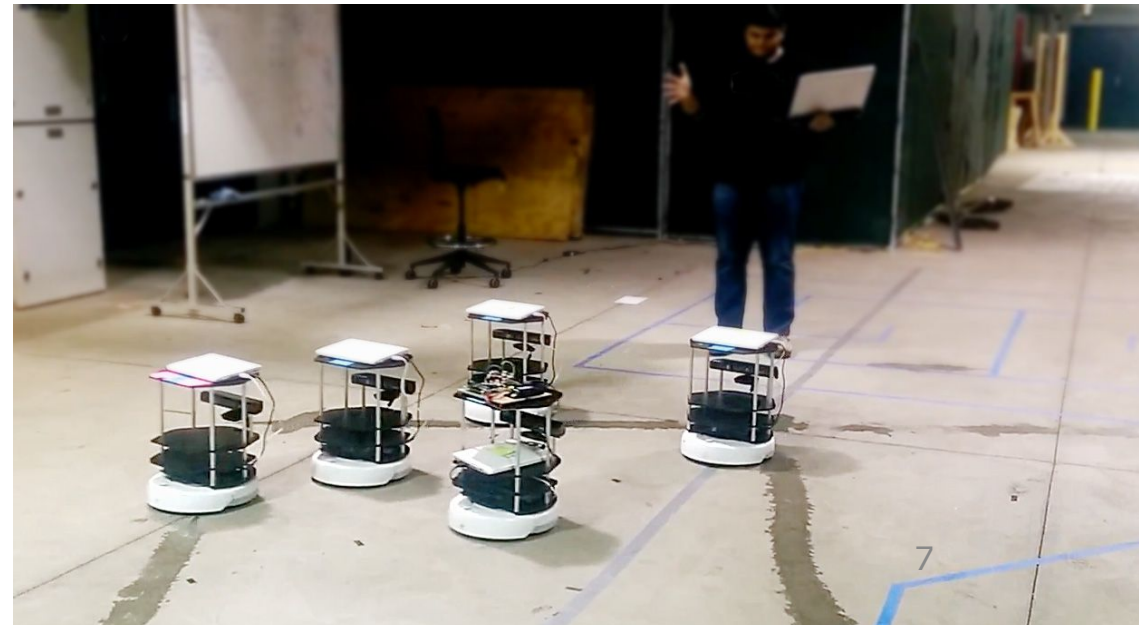




Comms-Constrained

Multiaagent Search

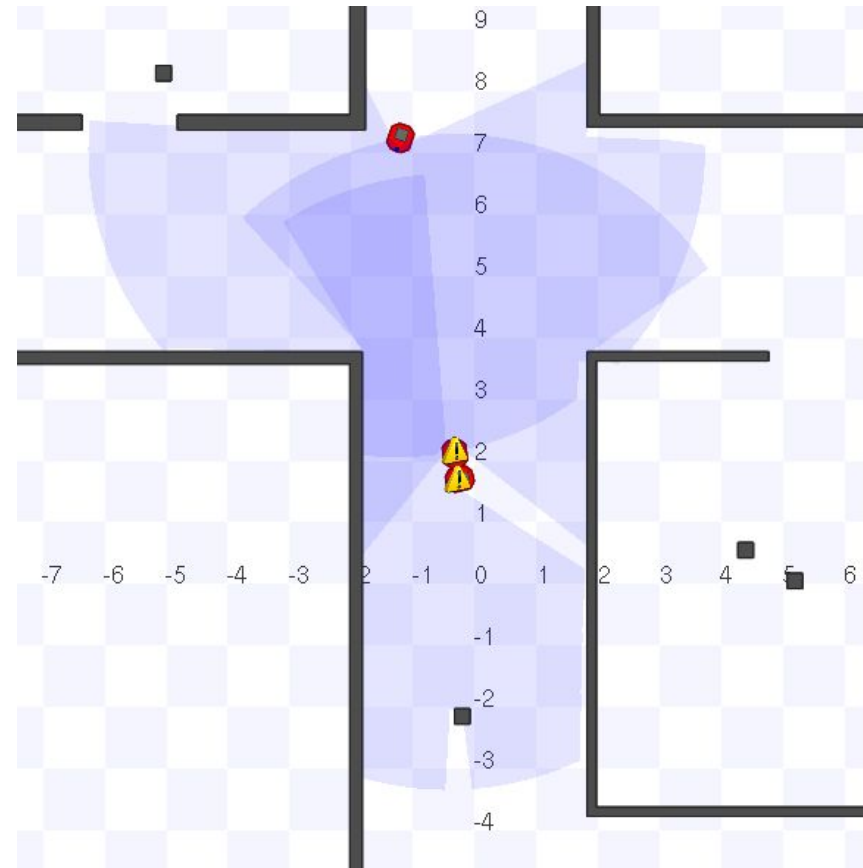
- Implementing multiagent has some other overheads not typical of single robot applications
 - Decentralized / Centralized
 - Juggling more than 1 navstack...





Comms-Constrained Multiaгент Search

- Coordinated group effort to minimize time to find two randomly placed targets
- Targets are randomly placed in a generated 2.5D simulated office space





Rubric



- Generates office environments - 8/8
- Loads simulator and runs a single agent - 8/8
- Communication system limits messages based on proximity - 8/8
- Finds a single target in some baseline environment with a single agent - 3/3
- Finds multiple targets in some baseline environment with multiple agents - 3/3

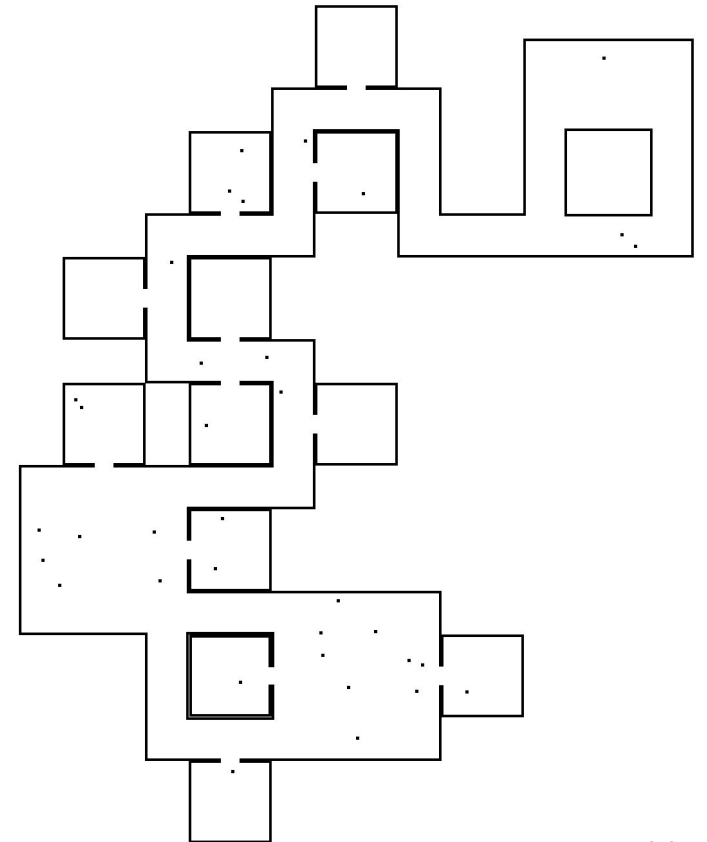


What We Built

1. Procedural Environment Generator
 - a. Generates randomized office-style floorplans and .world files for Stage at launch time
2. Distance-Based Communication Model
 - a. Node enforcing discrete communication zones and facilitating belief/map fusion
3. Multi-Agent Search Testbed
 - a. Framework handling localization, navigation, target detection, belief tracking, and map fusion automatically

Floorplan Generation - Backend

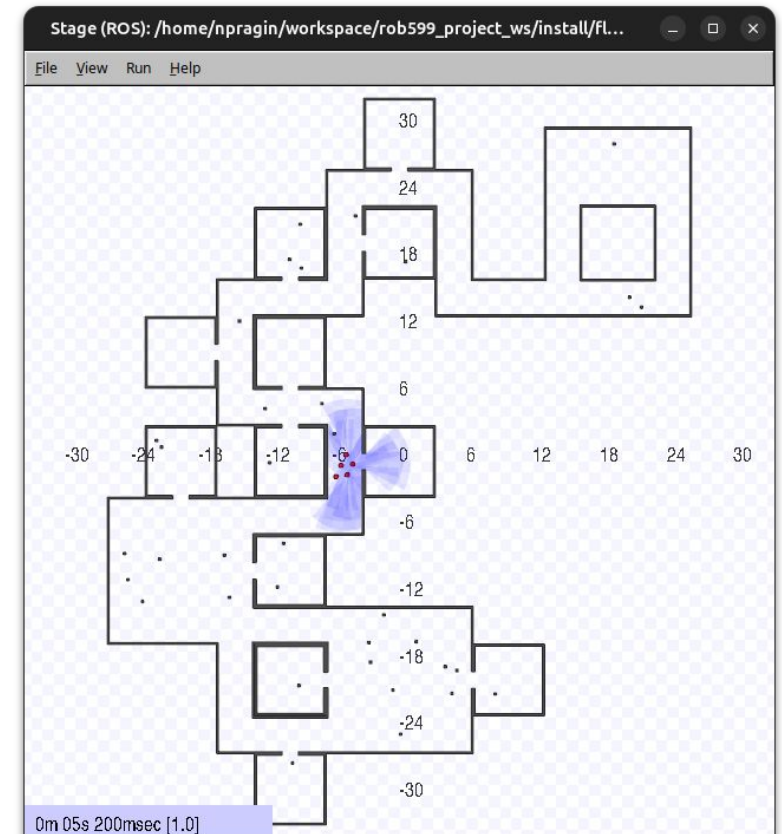
- Generates randomized-office style layouts
- Outputs a PNG
- Fully Configurable
 - Room density
 - Layout complexity (number of turns)
 - Room size
 - Obstacle density





Floorplan Generation - Stage

- Ingests generated PNG from backend and uses it with a templated .world file
- Places robots at valid locations within the environment
- Why procedural generation?





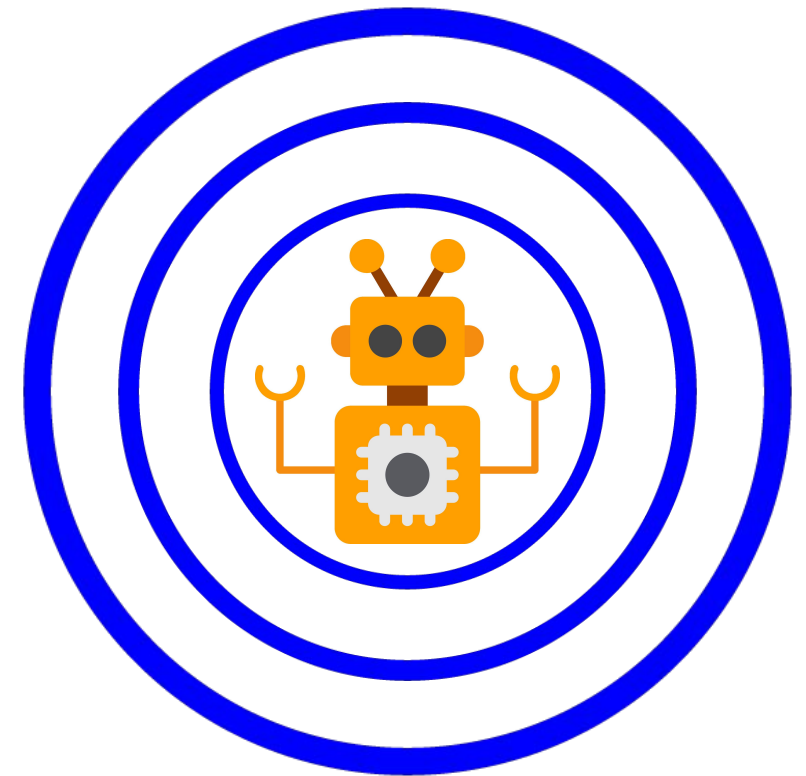
Distance-Based Communication Constraints

Zone	Distance	What's Delivered
Fusion Range	≤ 5 m & LOS	Belief/map fusion
Close Range	≤ 5 m	Coordination messages + heartbeats @ 10 Hz
Long Range	5-20 m	Heartbeats @ 1Hz
Blackout	> 20 m	Nothing



Distance-Based Communication Constraints

- Heartbeats carry peer pose estimates
- Coordination messages are arbitrary agent-defined structs
- All thresholds, rates, and cooldowns are configurable





Belief Tracking & Map Fusion

- Each agent maintains two internal grids:
 - **Belief** - log-odds over the target position
 - **Eliminated** - boolean of confirmed target-free cells
- When robots enter fusion range:
 - Maps are fused using a bayesian update
 - Beliefs are fused using a bayesian update
 - Eliminated cells are fused using a union



Per-Robot Autonomy Stack

- AMCL
 - Particle filter in known map mode
- slam_toolbox
 - Online SLAM in unknown map mode
- Nav2
 - Path planning and control
- Target Detector
 - Monitors LIDAR, calls service





The AgentBase Interface

- Algorithms are implemented by subclassing AgentBase, which handles everything for you

```
class ExampleAgent(AgentBase):
    """
    Example Agent for testing the multi-agent search system.

    Some other functions you might use:
    - publish_heartbeat
    - publish_coordination_message
    - navigate_to
    - cancel_navigation

    Some useful attributes for you to access:
    - self.agent_id
    - self.current_pose
    - self.belief
    - self.eliminated
    - self.map
    - self.use_known_map
    - self.known_initial_poses
    - self.map_info
    - self.nav_status
    """

    def __init__(self) -> None: ...

    def on_heartbeat(self, msg: HeartbeatMessage) -> None: ...

    def on_coordination(self, msg: BaseCoordinationMessage) -> None: ...

    def on_lidar_scan(self, scan: LaserScan) -> None: ...

    def on_target_detected(self, target_locations: list[tuple[float, float]]) -> None: ...

    def on_map_updated(self) -> None: ...

    def on_fusion_completed(self) -> None: ...

    def on_navigation_feedback(self, feedback: NavigateToPose.Feedback) -> None: ...

    def on_navigation_succeeded(self) -> None: ...

    def on_navigation_failed(self, reason: str) -> None: ...
```



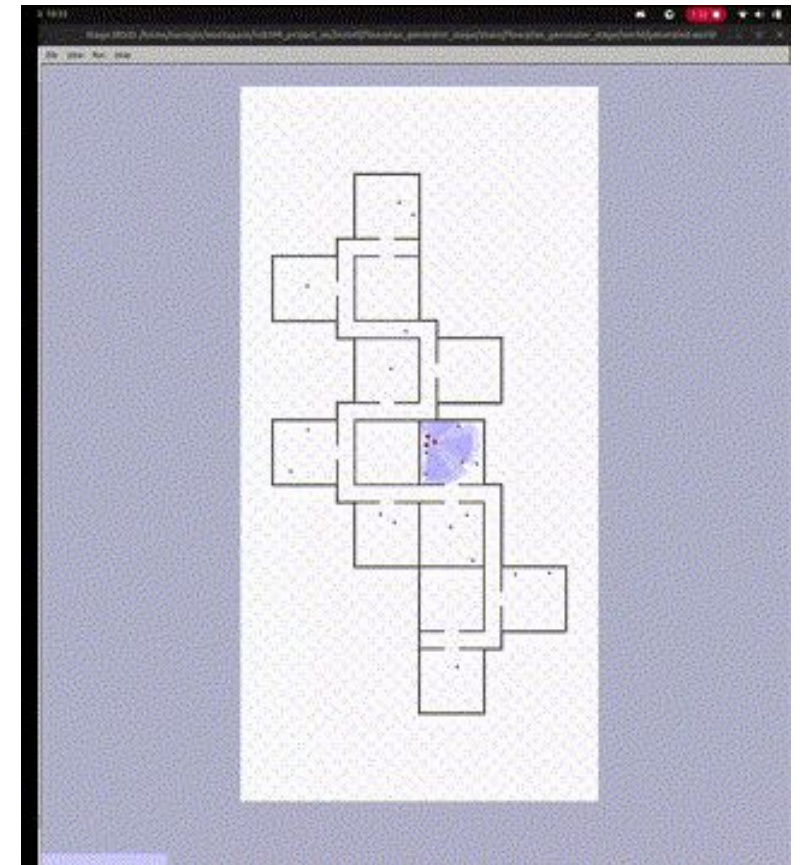
The Testbed

- Designed as a general-purpose testbed for future work on multi-agent search
 - Configurable # of robots and targets
 - Localization modes: Un/known map with un/known initial poses
- Example algorithms:
 - Uncoordinated frontier-based search
 - Convoy-based search



The Testbed - Metrics

- Success / Failure
- Time to first target
- Time to all targets
- Distance per agent
- Cumulative distance
- Map coverage (%) over time
- Redundant search area





Search Strategies Evaluated

1. Sequential Allocation of Rooms and Hallway Junctions
 - a. Graph of rooms and junctions, sequential auction
2. Voronoi-Based Exploration
 - a. Robots scatter, partition the map, and rendezvous opportunistically
3. Greedy Idle-based Policy
 - a. Coarse grid, implicit coordination via heartbeats, go to last visited neighbor



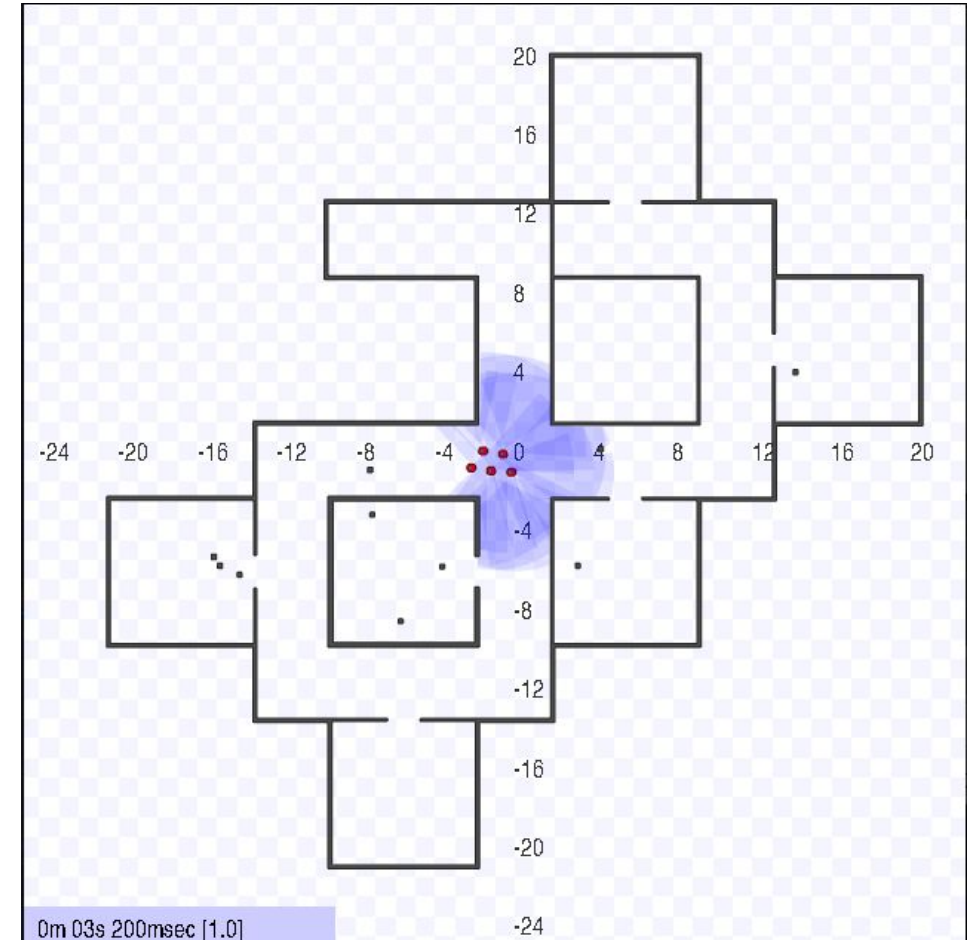
Experimental Setup

- Easy vs Hard Maps
 - Rooms, turns, hallway width, obstacle count
- 3 vs 5 Robots
- 5 separately seeded trails

Results: 3 Robots, Easy Map

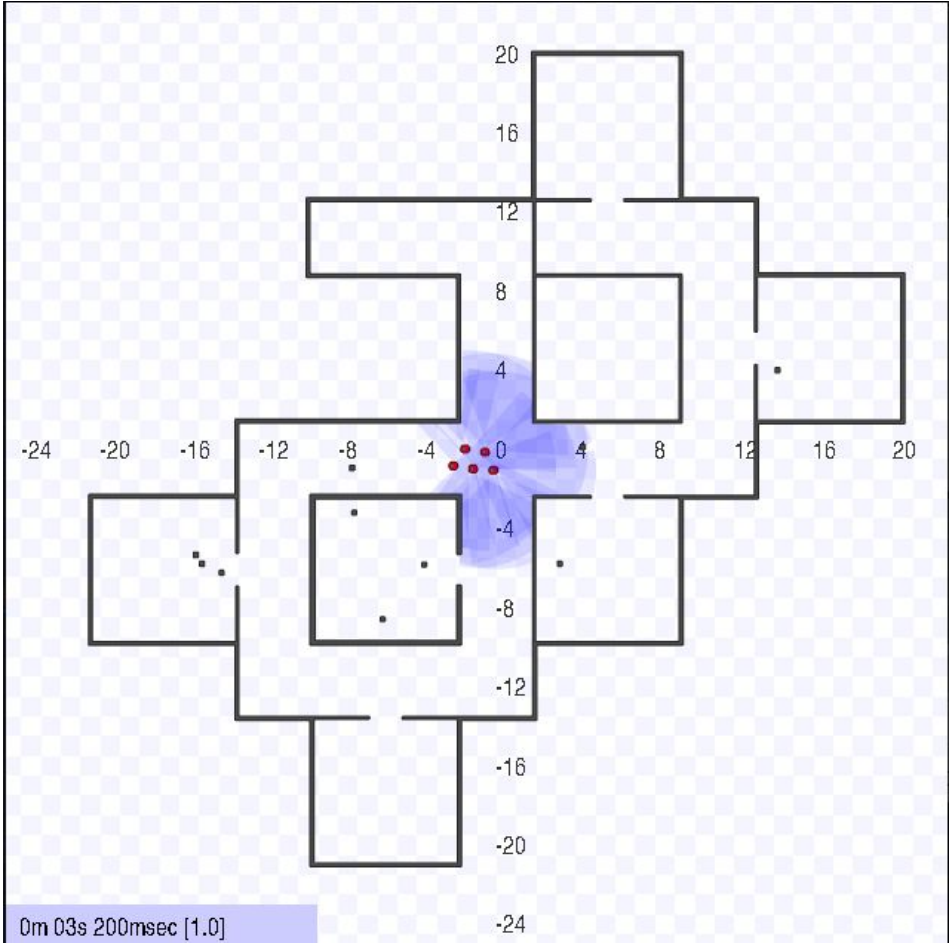


Algorithms	Time to First Target (seconds)	Time to All Targets (seconds)	Cumulative Distance (m)
Sequential	80	167	91
Voronoi	43	195	159
Idle-Based	165	364	214



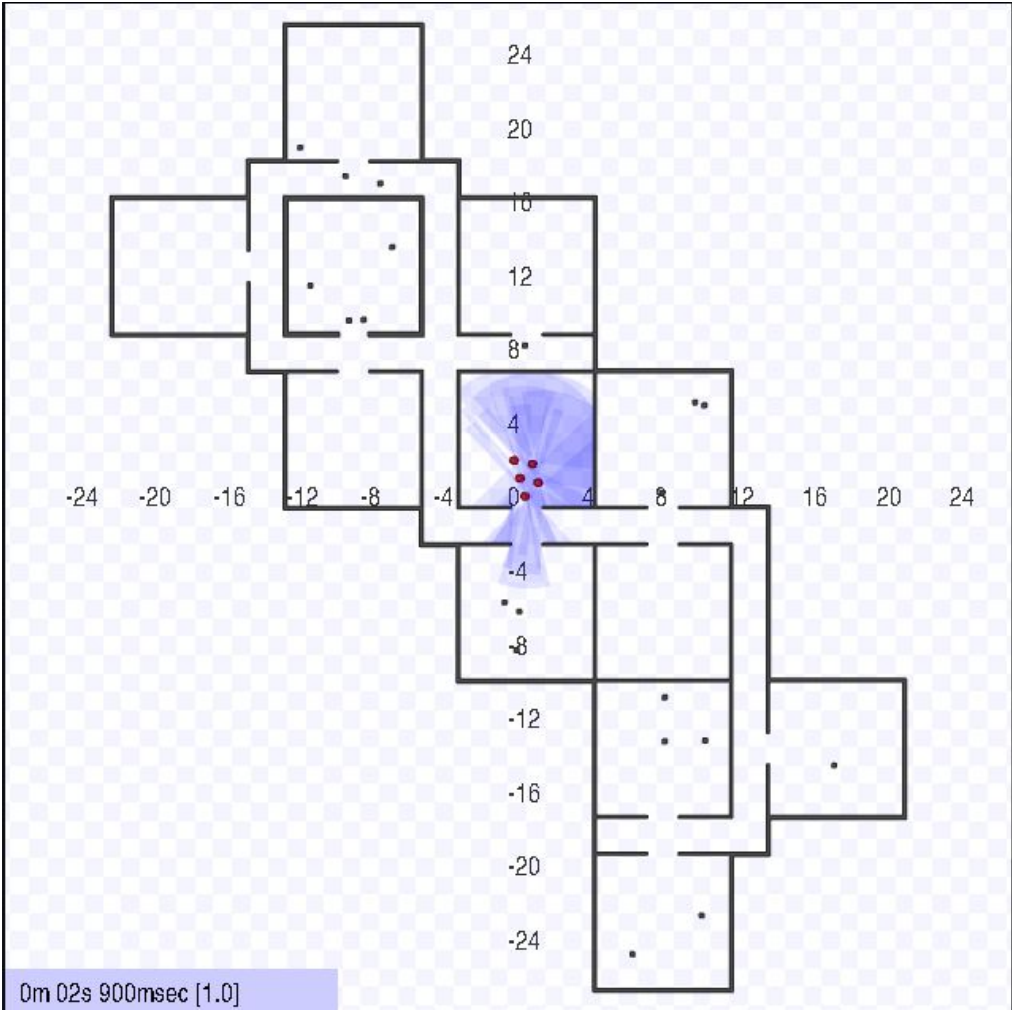
Results: 5 Robots, Easy Map

Algorithms	Time to First Target (seconds)	Time to All Targets (seconds)	Cumulative Distance (m)
Sequential	103	150	105
Voronoi	59	117	136
Idle-Based	64	135	292



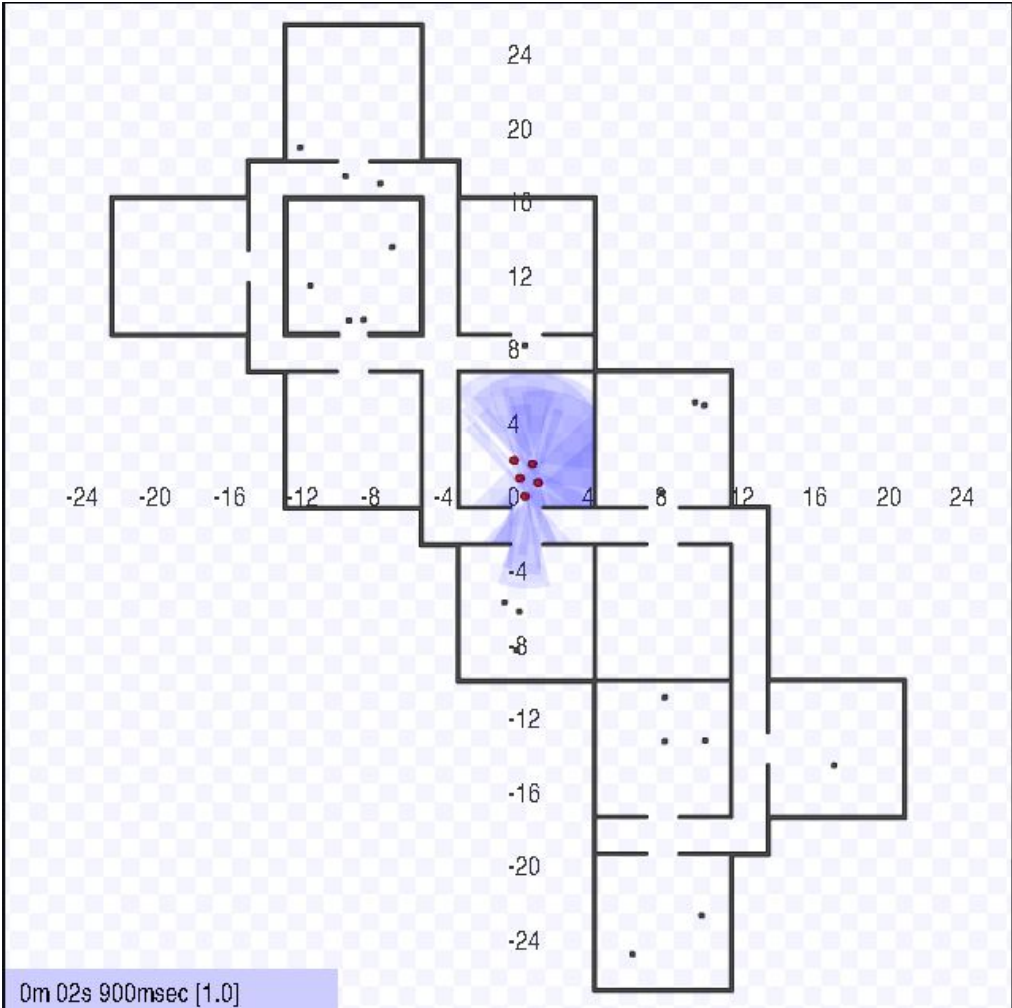
Results: 3 Robots, Hard Map

Algorithms	Time to First Target (seconds)	Time to All Targets (seconds)	Cumulative Distance (m)
Sequential	149	335	243
Voronoi	185	621	402
Idle-Based*	110*	270*	210*



Results: 5 Robots, Hard Map

Algorithms	Time to First Target (seconds)	Time to All Targets (seconds)	Cumulative Distance (m)
Sequential	187	291	202
Voronoi	86	434	449
Idle-Based	274	275	315





Observations

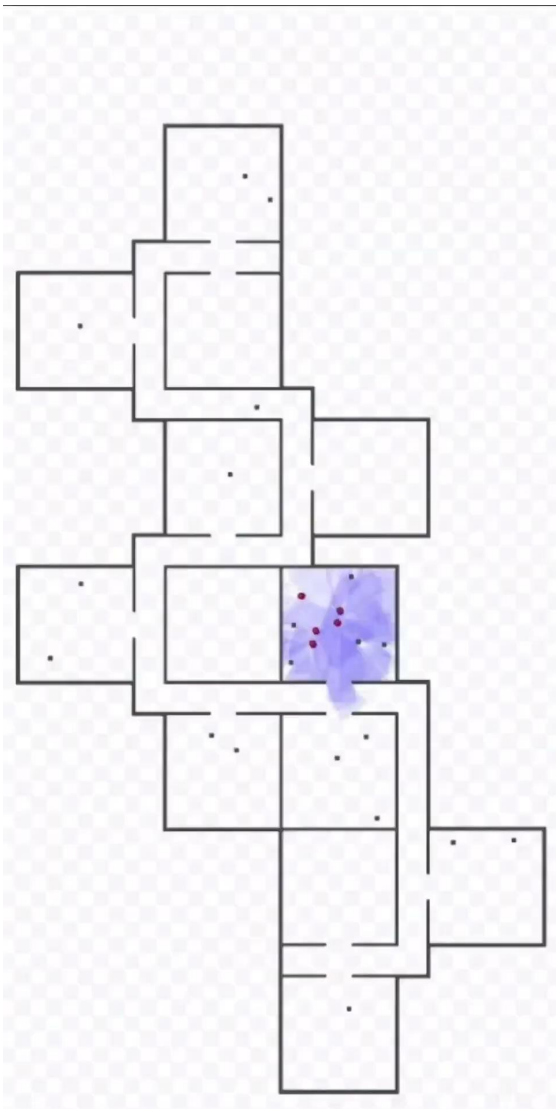
- Algorithms were much faster on the easy maps
- Voronoi often had the fastest time to first target
- Sequential often had the smallest cumulative distance
- 5 robots often found the goals faster than 3 robots

Results

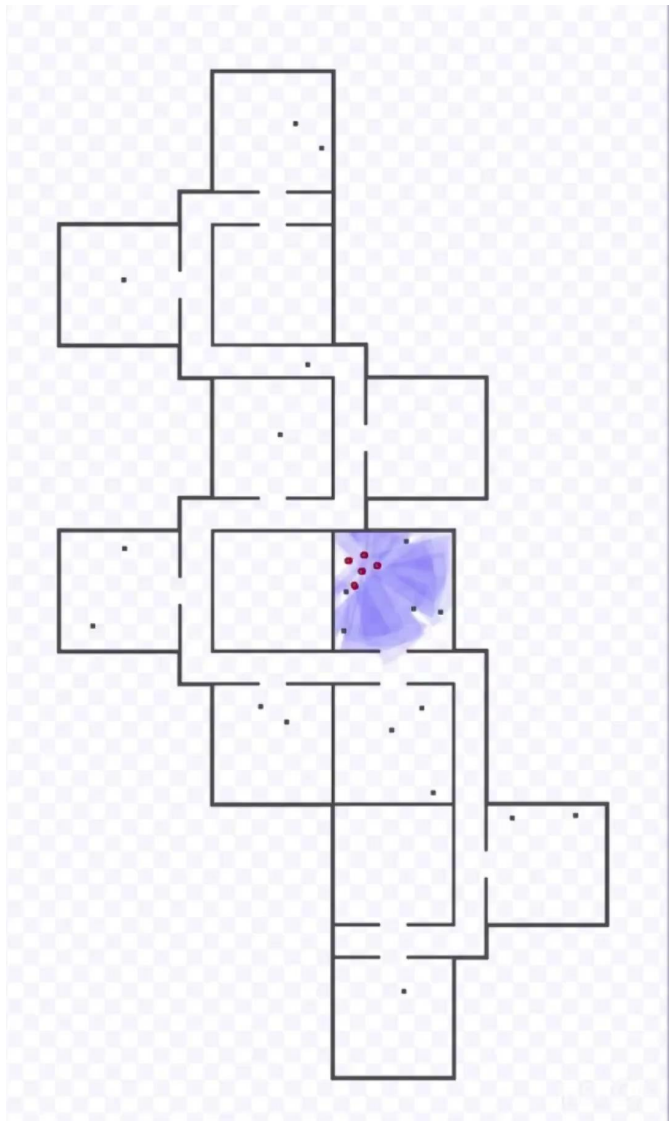
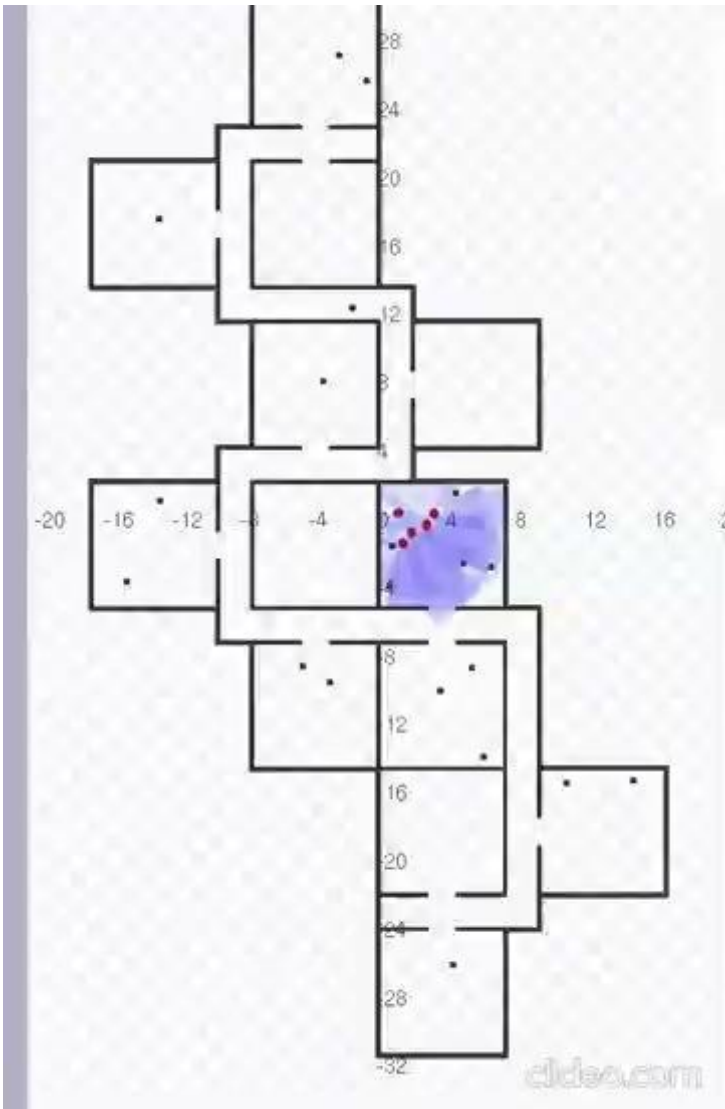


Oregon State University
College of Engineering
Greedy Idle-Based

Sequential Allocation



Voronoi-Based





Oregon State University
College of Engineering

Questions?

Comms-Constrained Multi-Agent Search

Noah Pragin

praginn@oregonstate.edu

Conor Gagliardi

gagliarc@oregonstate.edu

Liam Bouffard

bouffarl@oregonstate.edu

1 Overview

Multi-agent search is a well established field that is the logical succession of single-agent search, where adding additional agents expands the problem from planning only, to include coordination. The addition of coordination as a constraint raises the consideration of practical application in field robotics. To communicate, agents must use some form of wireless networking architecture. In practice, wireless communication is prone to many fail-states such as range limitations and line-of-sight in addition to the constraints of bandwidth and frequency for feasible messages. Many simulation environments tend to overlook this, assuming full connectivity, which hinders the sim-to-real capabilities.

Our project investigates multi-agent search in indoor environments under realistic communication constraints, using teams of three and five Robot Operating System (ROS2)-controlled LiDAR-equipped robots navigating procedurally generated office buildings to locate multiple static targets. The simulation stack uses the open source Stage simulator for the environment and ROS2-native Nav2 SLAM and waypoint path following for navigation, localization, and control. A central communication manager enforces three distance-based zones: full map sharing at close range, messaging at long range, and blackout beyond a threshold. Agents maintain Bayesian probability maps over target locations and asynchronously merge collaborative SLAM data when in close range. We implemented 3 separate coordinated search algorithms for comparison and evaluated performance across multiple experimental trials.

We chose this project because our team has preliminary experience when it comes to single-agent robotics applications, so the natural next step is to experiment with multiple agents. We were also interested in gaining experience with communication constrained coordination, as well as the practical knowledge of simulating it. Finally, this project is closely related to all team members' research and areas of interest.

Experiments with communication constrained robotics allows for real world usable coordination architectures that enable teams of robots to conduct difficult missions in degraded environments, where their help is most substantial. This may include burning buildings for survivor search, dangerous areas that may have hazards to human life, and broad-scale environments for search and rescue operations. Additionally, enabling sim-to-real deployment of communication-constrained multi-agent systems is a underexplored domain. By facilitating this pipeline we substantially decrease the effort required for iterative testing and development of future multi-agent robotics research. This contribution helps not only us, but the research community as a whole.

2 Initial Timeline and Revisions

Table 1: Project timeline. = initial plan, = revised plan, = unchanged.

Task	Wk 3	Wk 4	Wk 5	Wk 6	Wk 7	Wk 8	Wk 9	Wk 10
Multi-Agent Lit. Review	 							
Environment & Simulation		 	 					
ROS2 Architecture		 		 	 	 		
Communication System		 		 				
SLAM Integration			 			 	 	
Target Detection & Belief Updates			 		 			
Algorithm Implementation				 	 	 	 	

The initial weeks of this project focused on project specification and refinement. Each member of our team had related but different project goals. The cohesion of our projects required debate, voting, and compromise. These decisions were dynamic and slowly changed as we gained additional information during the literature review. To meet our objectives in a timely manner and create an environment of accountability, we delegated specific tasks to each team member. Due to external obligations such as research and paper writing for conferences, some tasks were backlogged to later weeks. These were the main discrepancies from our original plan.

3 Results

3.1 What We Built

Our implementation consisted of three major components: a procedural environment generator, a distance-based communication model, and a multi-agent search testbed. Each was designed to be independently useful and reusable, but together they form a complete system for developing and quantitatively comparing comms-constrained multi-agent search algorithms in simulation.

3.1.1 Procedural Environment Generator

The environment generator is a two-part pipeline for producing novel office-style simulation environments at runtime. The first part is a standalone Python library ([ros-floorplan-generator](#)) that uses Shapely for geometric polygon operations to procedurally generate randomized floorplans consisting of rooms, hallways, walls, and obstacles. Generation is fully configurable via a TOML file, exposing parameters such as room density, layout complexity, room size, and obstacle density. The output is a PNG occupancy grid image along with associated metadata.

The second part ([floorplan-generator-stage](#)) ingests that PNG and uses it alongside a templated Stage world file to produce a complete simulation environment, placing robots at valid spawn locations within the generated map. The entire pipeline runs automatically at launch, so every trial executes in a previously unseen environment. This design prevents any implicit overfitting to a fixed map and ensures that performance differences between algorithms reflect genuine behavioral differences rather than environment-specific tuning.

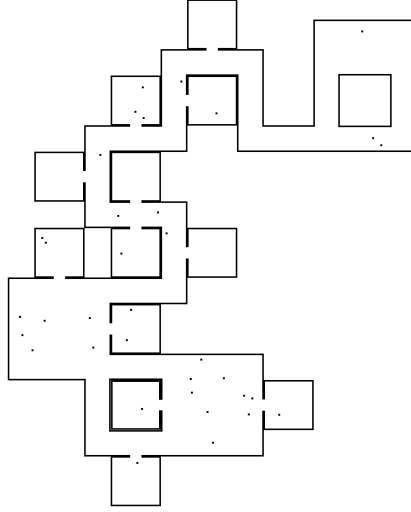


Figure 1: An example floorplan PNG.

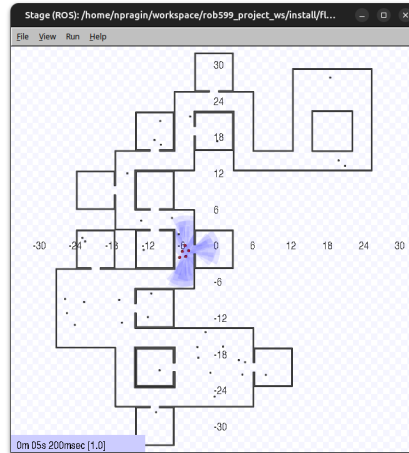


Figure 2: An example Stage world using the PNG shown in Figure 1.

3.1.2 Distance-Based Communication Model

The communication model is implemented as a ROS2 lifecycle node that mediates all inter-agent communication by enforcing discrete distance-based zones using ground truth positions from the simulator. Four zones are defined: a fusion range (≤ 5 m with line of sight) where agents exchange full map and belief data, a close range (≤ 5 m) where coordination messages and heartbeats are delivered at 10 Hz, a long range (5–20 m) where only heartbeats are delivered at 1 Hz, and a blackout zone (> 20 m) where no communication occurs. All thresholds, rates, and cooldowns are configurable via node parameters, enabling controlled experiments across different communication regimes without modifying agent code.

Two message types are available to agents. Heartbeats carry a peer's current pose estimate and are constructed and delivered automatically by the framework. Coordination messages are arbitrary agent-defined data structures that subclass a `BaseCoordinationMessage` base class giving algorithm implementations full flexibility to communicate whatever state is relevant to their coordination strategy.

3.1.3 Multi-Agent Search Testbed

The testbed (`multi-agent-search`) is a ROS2 framework that handles the full per-robot autonomy stack automatically, letting algorithm implementations focus exclusively on search and coordination logic. Each robot runs AMCL for particle filter localization in known map mode, or `slam_toolbox` [1] for online SLAM in unknown map mode, though full SLAM support is still under active development and was not used in our experiments. Navigation is handled by Nav2 [2] and a `TargetDetector` node monitors each robot’s lidar scans and fires a callback when a target is within a configurable detection radius, emulating a separate module on the robot. All of this runs under a per-robot namespace, allowing arbitrary team sizes configured at launch.

Algorithms are implemented by subclassing an `AgentBase` class that exposes hooks for relevant events: receiving a heartbeat or coordination message, processing a lidar scan, detecting a target, completing map fusion, and receiving navigation results and feedback. The base class also maintains per-agent state including the current occupancy map, a log-odds belief grid over target positions, and a boolean eliminated grid of confirmed target-free cells. When two agents enter fusion range, their maps and belief grids are fused via Bayesian updates and their eliminated grids are merged via union, automatically propagating shared exploration progress without any coordination logic required from the algorithm.

To validate the testbed, we implemented two search algorithms: an uncoordinated frontier search, and a frontier-based convoy search. We also implemented three sophisticated search and coordination algorithms to compare against each other. The first, sequential allocation of rooms and junctions, builds a graph of rooms and hallway corners and assigns them to robots via a sequential auction. The second, Voronoi-based exploration, initializes by scattering the robots, partitions the space by generating a Voronoi diagram, with robots rendezvousing opportunistically to share belief state. The third, a greedy idle-based policy, overlays a coarse grid on the environment and uses implicit coordination leveraging heartbeat messages to direct the robot to the least recently visited neighboring cell.

The testbed automatically collects seven metrics across all trials: success/failure, time to first target, time to find all targets, per-agent distance traveled, cumulative team distance traveled, map coverage (%) over time, and redundant search area. Time and distance can be synthesized into human-interpretable metrics such as battery consumption. Redundant search area is particularly relevant to evaluating coordination quality under communication constraints, since poorly coordinated teams tend to re-search areas that teammates have already cleared.

3.2 Experimental Results

Within this section we present the results of our findings. Four tables are presented, each displaying the data tied to a unique independent variable combination of an easy or hard map and three or five robots. The objective for these trails is to locate two randomly placed goal markers within the map within fifteen minutes. If a trail’s duration persists longer than fifteen minutes, it’s marked as Miss and not included within the metric calculations. For each table, the three algorithms, Idle-Based, Sequential, and Voronoi-Based Partitioning, are tested with 5 uniquely seeded trials. We present five dependent variables to capture the success of each algorithm: Time to First Goal (T1), Time to All Goals (TAI), Distance Traveled (Dist), Coverage Percentage (Cov), and Redundant Coverage Percent (Red). Time to First Goal measures the number of seconds till at least one robot has one of the goals within its lidar. The Time to All Goals measures the amount of seconds passed until all goals, in our case two, have been picked up by the lidar. The third dependent variable, Distance Traveled, measures the total distance moved (meters) for all robots, providing an insight into how active each algorithm was before trail termination. Coverage Percentage measures how much of the map was traversed by the team of robots in their pursuit. Finally, Redundant Coverage Percent tracks the efficiency of how well the team of robots avoids previously searched regions.

Table 2: Easy Map, 3 Robots (T1: Time to First Goal, TALL: Time to All Goals, Dist: Distance Covered, Cov%: Coverage Percent, Red%: Redundant Coverage Percent, μ : mean, σ : std)

Alg	T1			TALL			Dist			Cov%			Red%		
	μ	σ	Miss	μ	σ	Miss	μ	σ	Miss	μ	σ	Miss	μ	σ	Miss
Idle	138	166	0	304	234	0	215	166	0	56	25	0	67	7	0
Seq	80	76	0	167	47	0	91	60	0	56	25	0	38	13	0
Vor	44	42	0	196	111	0	159	88	0	57	23	0	60	13	0

Table 3: Hard Map, 3 Robots (T1: Time to First Goal, TALL: Time to All Goals, Dist: Distance Covered, Cov%: Coverage Percent, Red%: Redundant Coverage Percent, μ : mean, σ : std)

Alg	T1			TALL			Dist			Cov%			Red%		
	μ	σ	Miss	μ	σ	Miss	μ	σ	Miss	μ	σ	Miss	μ	σ	Miss
Idle	160	19	1	270	42	3	210	93	0	51	22	0	50	24	0
Seq	147	133	0	335	111	0	243	81	0	74	18	0	46	15	0
Vor	185	337	0	621	375	0	403	297	0	68	23	0	64	19	0

Table 4: Easy Map, 5 Robots (T1: Time to First Goal, TALL: Time to All Goals, Dist: Distance Covered, Cov%: Coverage Percent, Red%: Redundant Coverage Percent, μ : mean, σ : std)

Alg	T1			TALL			Dist			Cov%			Red%		
	μ	σ	Miss	μ	σ	Miss	μ	σ	Miss	μ	σ	Miss	μ	σ	Miss
Idle	80	88	1	138	121	1	292	295	0	53	30	0	73	11	0
Seq	103	61	0	151	63	1	105	50	0	61	29	0	55	12	0
Vor	59	64	0	117	110	0	136	140	0	52	32	0	66	15	0

Table 5: Hard Map, 5 Robots (T1: Time to First Goal, TALL: Time to All Goals, Dist: Distance Covered, Cov%: Coverage Percent, Red%: Redundant Coverage Percent, μ : mean, σ : std)

Alg	T1			TALL			Dist			Cov%			Red%		
	μ	σ	Miss	μ	σ	Miss	μ	σ	Miss	μ	σ	Miss	μ	σ	Miss
Idle	274	506	1	275	63	2	316	41	0	49	12	0	54	20	0
Seq	187	109	0	291	57	0	202	59	0	69	17	0	59	13	0
Vor	89	97	0	434	210	0	449	206	0	72	10	0	70	10	0

4 Discussion and Learnings

4.1 Discussion

Through experimental trials we noticed an interesting development. Often times Voronoi-based exploration were the first to find the first target, but had some inefficiencies due to the rendezvous policy that did not scale well with environment size. Sequential Allocation of Rooms and Hallway Junctions algorithm tended to produce the shortest cumulative distance, indicating that it was the most resource efficient strategy. Finally, on larger maps, a Greedy Idle-based Policy tended to have to best time to find all targets, indicating that it was the best scaling strategy. These results may be skewed due to limited experimental trials which were chosen based on remaining time in the quarter.

It was noted that the five robot case had, on average, a lower time to reach both goals when compared to the 3 robot case for both the easy and hard maps. This was expected due to the five robot's ability to cover more ground more efficiently. However, the differences in the values weren't as significant as one would expect. From observation, although a larger number of robots increases coverage efficiency, it can also increase the probabilities of robot congestion. The occurrence of this problem is most evident in the beginning of the search event where the robots start near to one another. The frequency of this is also dependent on the width of the hallway and rooms, which directly correlated to the robot's ability to successfully pass one another.

Additionally, it was observed that the easy maps had a significantly lower time for achieving both goals than the hard maps. Between the two room difficulties, three independent variables were

changed to vary the difficulty. The easy map has a total of six room, a 3.6 meter hallway width, and an obstacle count of 10 which are 0.2 meters wide. The hard map, in contrast, has twelve rooms, a 1.8 meter hallway width, and 20 obstacles of the 0.2 meters wide. Increasing the room count has a direct proportional affect on the amount of total area required for searching. As the required area increases, so does the time it takes for the robots to search. Decreasing the hallway width increases the probability of congestion, which contributes to the challenge of the task. Finally, an increase in obstacles creates difficult to navigate terrain and decreases each robots visibility when searching for the target. Based on the data, it's clear this contrast between easy and hard maps directly results in change in goal task completion time.

Here we'll discuss further observations. The Sequential algorithm had the lowest amount of Distance Covered for 3/4 of the tables. This can be likely attributed to the how the underlying graph of this algorithm is developed. In the Sequential case, each room exists as a node, and the paths between as edges. Thus, the direct movement between room nodes, without idling within hallway would likely create the results described. The Coverage Percent among the algorithms were roughly similar, with no particular algorithm being exceedingly more efficient then another. Finally, the Sequential algorithm had the lowest Redundant Coverage Percent, likely due to similar reasons as the Distance Covered metric.

4.2 Learnings

We learned that extending ROS operation from single to multiagent applications introduces an unexpected issue of native DDS broadcasting floods. Using Nav2 for 3 separate agents within the same network resulted in difficulty initializing all agents simultaneously. Fortunately, we learned how to implement Zenoh to remediate these issues.

5 Future Work

5.1 Testbed Infrastructure

The most immediate infrastructure gap is completing unknown map mode. The current system supports SLAM-based localization via `slam_toolbox` as a configuration option, but consistent belief updatation across dynamically built maps is not yet stable enough for experimental use. Closing this gap would unlock a significantly harder and more realistic problem setting where agents cannot rely on a pre-existing map for planning or localization.

The floorplan generator currently produces office-style environments with a main hallway and rooms branching off of it. While loops can appear in generated layouts, they arise incidentally from the generation process rather than being an explicit topological primitive. Hallway-dominant layouts may systematically favor certain coordination strategies and make results less generalizable. A planned extension is to introduce explicit topology control, allowing the generator to produce qualitatively distinct environment types such as grid layouts, hub-and-spoke arrangements, or a single large loop.

Finally, robot spawn positions are currently clustered, with no support for randomization across the environment. Randomizing initial poses would better stress-test coordination strategies that depend on early rendezvous or shared initialization.

5.2 More Realistic Problem Formulation

The current system assumes that target detection is immediate and certain once a robot is within the detection radius, and that a single agent's observation is sufficient to confirm a target. A natural first extension is to require multiple agents to independently observe and agree on a detection before it is confirmed, more closely modeling real verification requirements. Related to this, removing the assumption that agents can instantly recognize a target and replacing it with a probabilistic or time-integrated detection model would introduce the need for agents to reason about detection uncertainty.

The communication model currently applies hard cutoffs at zone boundaries. A more realistic formulation would replace these with continuous degradation, throttling publish rates as a function of distance rather than switching behavior discretely at fixed thresholds.

The most substantial extension to the problem formulation is introducing target movement and evasion. In this setting, targets move and actively attempt to avoid detection, requiring agents not only to search unseen areas but to maintain coverage of previously cleared regions to prevent targets from backtracking into them. This transforms the problem from a one-shot coverage task into a persistent surveillance problem, substantially increasing its difficulty and practical relevance.

5.2.1 Physical Deployment

Although our project is meant to provide a clear understanding of coordination and search algorithm failure modes and weaknesses to guide design decisions, the logical endpoint of this work is deploying the algorithms on physical robots with real sensors. Transitioning from Stage simulation introduces sensor noise, localization drift, actuator imprecision, and true communication constraints that cannot be fully captured in simulation, and would serve as the ultimate validation of algorithms developed in this testbed.

References

- [1] S. Macenski and I. Jambrecic. Slam toolbox: Slam for the dynamic world. *Journal of Open Source Software*, 6(61):2783, 2021.
- [2] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall. Navigation2: The next generation navigation system for ros. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2022.